Chapter 8: Monorepo Project Structure

Welcome back to the CinéConnect tutorial! In Chapter 7: API Request Validation, we learned how `Zod` acts as a strict bouncer, ensuring all incoming API requests are valid and secure. Now that we understand how individual pieces of our application work, let's zoom out and look at the bigger picture: how `CinéConnect` itself is organized as a whole project.

🏢 One Big Library: Why Project Structure Matters

Imagine CinéConnect is a collection of several closely related "books." We have one book for the frontend (what you see and click), one for the backend (the server logic), and another smaller book full of shared recipes and tools that both the frontend and backend use.

You could store these books in separate, small libraries (many separate repositories). But what if these books often refer to each other or need to share common ingredients? Constantly running between different libraries to find what you need can be inefficient.

This is where a **"Monorepo"** comes in! It's like storing all these closely related books in *one big, well-organized library*. Everything is under one roof, making it easier to manage, share, and develop.

Our main goal in this chapter is to understand how `CinéConnect` is structured as a "monorepo" and how special tools like `pnpm workspaces` and `Turbo` help us manage all its parts efficiently, making development faster and simpler.

Let's explore this "one big library" concept with an example: **building our entire application.**

The Monorepo Team: `pnpm workspaces` and `Turbo`

In our CinéConnect "library," we have two expert assistants:

1. **`pnpm workspaces` - The Organizer**:
    - `pnpm` is our package manager (like `npm` or `yarn`), but `pnpm workspaces` is its special feature for monorepos.
    - It's like the main librarian who knows exactly where each "book" (our `apps` and `packages`) is located and what other "books" it needs (its dependencies). It makes sure all these connections are properly set up.
    - It allows us to install and manage dependencies for *all* parts of our project from one central place, while intelligently sharing common dependencies to save disk space and installation time.
2. **`Turbo` - The Smart Assistant**:
    - `Turbo` (short for `Turborepo`) is like a super-smart assistant librarian who makes tasks lightning-fast.
    - When you ask to "build all the books" or "test all the books," `Turbo` doesn't just blindly rebuild everything from scratch every time.
    - It intelligently remembers what it built before (`caching`). If you've only changed one "book," `Turbo` only works on that one book and any other books that depend on it. It skips everything

else, saving a lot of time!

Our Project Layout: `apps/` and `packages/`

In our CinéConnect monorepo, you'll see a clear structure:

```
cine-connect/
├── apps/            # Our main applications (like the frontend and backend)
│   ├── frontend/    # The user interface (React + Vite)
│   └── backend/     # The server logic (Express + TypeScript)
├── packages/        # Smaller, reusable tools or libraries
│   └── shared/      # Shared types and utilities used by both frontend and
backend
└── ... (other project files like README, pnpm-workspace.yaml)
```

- **`apps/`**: This directory holds our actual runnable applications.
  - `frontend/`: Our web application that users interact with.
  - `backend/`: Our server that handles data, authentication, and API requests.
- **`packages/`**: This directory holds smaller, independent pieces of code that can be shared across our `apps`.
  - `shared/`: This package contains common TypeScript types, utility functions, or constants that both `frontend` and `backend` might need. Instead of duplicating this code, we put it here, and both apps can "import" it.

How to Work with the Monorepo: Building the Entire Application

Let's use the example of **building the entire application**. When we make changes, we need to compile our code into a runnable format.

The Developer Experience

1. You make changes in both the `frontend` and `backend` code.
2. You want to build both parts of the application and the `shared` package efficiently.
3. You run a single command.

Using `pnpm` and `Turbo` Together

In `CinéConnect`, thanks to the monorepo setup, building everything is a single, simple command:

```
pnpm build
```

Input: You type `pnpm build` in your terminal from the root of the `cine-connect` project.

Output (High-level):

1. `pnpm` sees the `build` script in the root `package.json` (which usually calls `turbo run build`).
2. `Turbo` takes over. It first checks if the `shared` package needs building.

3. Then, it builds `frontend` and `backend` projects.
4. If nothing has changed in `frontend` since the last build, `Turbo` will smartly skip building it again and use a cached version, making the process much faster!

This single command intelligently coordinates building all parts of our application, respecting their dependencies (e.g., `frontend` and `backend` both depend on `shared`, so `shared` must be built first) and leveraging caching for speed.

Under the Hood: The Monorepo in Action

Let's see how `pnpm workspaces` and `Turbo` work together when you run `pnpm build`. Diagram: Monorepo Build Process

1. **Developer Runs Command**: You run `pnpm build` from the root.
2. **`pnpm` Delegates to `Turbo`**: The root `package.json`'s `build` script tells `pnpm` to execute `turbo run build`.
3. **`Turbo` Orchestrates**: `Turbo` looks at all projects (`shared`, `frontend`, `backend`) and their `build` scripts. It also knows their dependencies.
4. **Dependency Order**: `Turbo` ensures that the `shared` package is built *before* `frontend` or `backend` (because they depend on it).
5. **Caching Magic**: For each project, `Turbo` checks its cache.
    - If a project (e.g., `frontend`) hasn't had any relevant files changed since its last successful build, `Turbo` *skips* the actual build process and just reports that it's "cached." This is a huge time saver!
    - If a project has changed, `Turbo` executes its `build` script.
6. **Completion**: Once all projects are built (or served from cache), `Turbo` reports success, and your `pnpm build` command finishes.

Diving into the Code: The Monorepo Configuration

Let's look at the minimal code snippets that enable this monorepo structure.

1. Defining the Workspaces (`pnpm-workspace.yaml`)

This file tells `pnpm` which directories are part of the monorepo's "workspaces."

```
# pnpm-workspace.yaml
packages:
  - 'apps/*'
  - 'packages/*'
```

- `packages: ['apps/*', 'packages/*']`: This is the core configuration. It tells `pnpm` to look inside the `apps` directory and the `packages` directory. Any subdirectories found within these (like `apps/frontend`, `apps/backend`, `packages/shared`) are considered individual "workspaces" or "projects" within the monorepo. `pnpm` will then manage their dependencies and linking.

2. Root `package.json` Scripts (`package.json`)

Our main `package.json` at the project root defines common commands that leverage `Turbo`.

```
{
  "name": "cine-connect",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "turbo run dev",
    "build": "turbo run build",
    "lint": "turbo run lint",
    "typecheck": "turbo run typecheck",
    "test": "turbo run test"
  },
  "dependencies": {
    "turbo": "^2.0.4"
  },
  "packageManager": "pnpm@9.4.0"
}
```

- `"private": true`: This indicates that the root of our monorepo itself is not meant to be published as a package.
- `"scripts": { ... }`: This defines common commands. Notice how almost all of them start with `turbo run <script-name>`. This tells `pnpm` to execute `Turbo`, which will then look for the `<script-name>` in *all* the individual projects (`frontend`, `backend`, `shared`) and run them intelligently.
    - `"build": "turbo run build"`: This is the command that executes the `build` script in each of our `apps` and `packages`.

3. Project-Specific `package.json` (`apps/frontend/package.json`)

Each individual project inside `apps/` or `packages/` has its own `package.json`.

```
// apps/frontend/package.json (Simplified)
{
  "name": "@cine-connect/frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build", // This is the 'build' script Turbo runs
    "lint": "eslint . --ext ts,tsx --report-unused-disable-directives --max-
warnings 0",
    "preview": "vite preview",
    "typecheck": "tsr generate && tsc -p tsconfig.app.json",
    "clean": "rm -rf dist .turbo"
  },
  "dependencies": {
    "@cine-connect/shared": "workspace:*", // Links to our shared package!
    // ... other frontend dependencies ...
  }
}
```

- `"name": "@cine-connect/frontend"`: This is the unique name for the frontend project within the monorepo.
- `"build": "vite build"`: This is the actual build command for the frontend. When `turbo run build` is executed from the root, `Turbo` will find this script and run it (if `frontend` needs building).
- `"@cine-connect/shared": "workspace:*"` : This is the magic of `pnpm workspaces`! It tells `pnpm` that `@cine-connect/frontend` depends on `@cine-connect/shared`, and `pnpm` should automatically link to the `packages/shared` project *within the same monorepo*. This ensures that if you change something in `shared`, `frontend` will use the latest version immediately.

4. `Turbo` Configuration (`turbo.json`)

`Turbo` also has its own configuration file, typically `turbo.json`, at the root of the monorepo. This file tells `Turbo` how to behave.

```
// turbo.json (Simplified)
{
  "$schema": "https://turbo.build/schema.json",
  "globalDependencies": [
    "**/.env",
    "**/.env.*",
    "tsconfig.json",
    "pnpm-lock.yaml"
  ],
  "pipeline": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": ["dist/**", ".next/**"],
      "env": ["VITE_API_BASE_URL", "VITE_APP_NAME", "VITE_TMDB_IMAGE_BASE_URL"]
    },
    "lint": {
      "dependsOn": ["^build"]
    },
    "typecheck": {
      "dependsOn": ["^build"]
    },
    "dev": {
      "cache": false,
      "persistent": true
    },
    "clean": {
      "cache": false
    }
  }
}
```

- `"pipeline"`: This is the heart of `Turbo`'s configuration. It defines how different scripts (like `build`, `lint`, `typecheck`) should behave across our workspaces.
- `"build"`:

    - `"dependsOn": ["^build"]`: This is crucial! It tells `Turbo` that to `build` a project (e.g., `frontend`), it must first `build` any projects it depends on (e.g., `shared`). The `^` means it looks at the dependencies of the current project.
    - `"outputs": ["dist/**", ".next/**"]`: This tells `Turbo` which files are produced by a `build` task. `Turbo` will cache these output files.
- `"dev"`:
    - `"cache": false`: `dev` servers are typically long-running, so we don't cache their output.
    - `"persistent": true`: Tells `Turbo` to keep the `dev` processes running.

This `turbo.json` file is what allows `Turbo` to be so intelligent about dependency ordering and caching.

Monorepo vs. Traditional Multi-Repo (Polyrepo)

Here's a quick comparison of the monorepo approach used by CinéConnect versus a traditional multi-repository setup:

Feature	Monorepo (CinéConnect)	Traditional Multi-Repo (Polyrepo)
Project Structure	All related projects in one repository.	Each project (frontend, backend, shared) in its own repository.
Code Sharing	Easy and natural (`workspace:*` dependencies).	Requires publishing packages, more complex version management.
Development	Single `pnpm install`, easier cross-project changes.	Multiple `git clone`, `npm install` for each project.
Build/Test	`Turbo` caches, runs tasks only on changed projects.	Must build/test each repository separately.
Version Control	Single `git` repository for everything.	Multiple `git` repositories.
Analogy	One big, organized library.	Many small, separate libraries.

For projects like CinéConnect, where the frontend, backend, and shared utilities are tightly coupled and often change together, a monorepo offers significant advantages in terms of development speed and maintainability.

Conclusion

In this chapter, we've unravelled the concept of `CinéConnect`'s monorepo project structure. We learned that a monorepo is like a single, organized library for all our closely related applications (`frontend`, `backend`) and shared tools (`shared` package). We saw how `pnpm workspaces` acts as the organizer, managing dependencies effortlessly, and how `Turbo` is our smart assistant, speeding up tasks like building and testing by intelligently caching and only working on what has changed. This powerful combination makes developing and maintaining `CinéConnect` faster and more efficient.

This marks the end of our tutorial chapters! You now have a solid understanding of the core concepts that power CinéConnect, from its user interface and authentication to data management, third-party integrations, real-time communication, backend architecture, and project organization.Next up: Testing Strategy!